

AI Assisted Coding

Assignment-12.3

Name: K. Sai Krishna Reddy

Rollno: 2303A52305

Batch: 45

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
 - o Quick Sort
 - o Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

- Correctly sorted student records.
- Performance comparison between Quick Sort and Merge Sort.
- Clear output of top-performing students.

The screenshot shows a VS Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named '2303A52305 AI A-C ASS-12.3' containing files 'AIAC_assignment_12.3_2305.docx', 'Task-1.py', 'Task-2.py', 'Task-3.py', 'Task-4.py', and 'Task-5.py'. The 'Task-1.py' file is open in the editor, showing a Python script that uses the dataclasses module to create a 'Student' class and functions to print student records and top performers. The terminal at the bottom shows the command to run the script and its output, which displays a table of student records.

```
1 import random
2 import sys
3 import time
4 from dataclasses import dataclass
5 from typing import List
6
7 @dataclass
8 class Student:
9     name: str
10    roll: str
11    cgpa: float
12
13
14 def print_students(students: List[Student]) -> None:
15     print("\nSorted Student Records:")
16     print(f"{'Name':<25}{ 'Roll Number':<15}{ 'CGPA':<6}")
17     for student in students:
18         print(f"{student.name:<25}{student.roll:<15}{student.cgpa:..2f}")
19
20
21 def print_top_students(students: List[Student], top_n: int = 3) -> None:
22     print(f"\nTop {min(top_n, len(students))} Performer{'s' if top_n != 1 else ''}:")
23     print(f"{'Name':<25}{ 'Roll Number':<15}{ 'CGPA':<6}")
24     for student in students[:top_n]:
25         print(f"{student.name:<25}{student.roll:<15}{student.cgpa:..2f}")
26
27
```

Terminal Output:

```
5 AI A-C Ass-12.3'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\ext
--
Sorted Student Records:
Name          Roll Number  CGPA
Ram           1234        9.00
Sai           2305        8.00
```

Task 2: Implementing Bubble Sort with AI Comments

- Task: Write a Python implementation of Bubble Sort.
- Instructions:
- Students implement Bubble Sort normally.
- Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
- Request AI to provide time complexity analysis.
- Expected Output:
- A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

```
1 def bubble_sort(arr):
2     # Get the number of elements in the list
3     n = len(arr)
4
5     # Repeat passes through the list until it is sorted
6     for i in range(n):
7         # Track whether any swaps occurred on this pass
8         swapped = False
9
10        # Compare each pair of adjacent elements in the unsorted portion
11        for j in range(0, n - i - 1):
12            # If the earlier element is larger, swap them to move the larger
13            # element toward the end of the list.
14            if arr[j] > arr[j + 1]:
15                arr[j], arr[j + 1] = arr[j + 1], arr[j]
16                swapped = True
17
18        # If no swaps happened, the list is already sorted and we can stop.
19        if not swapped:
20            break
21
22
23 # Sample data to sort
24 data = [64, 34, 25, 12, 22, 11, 90]
25
26 # Sort the list using bubble sort
27 bubble_sort(data)
28
29 # Print the sorted result
30 print("Sorted array:", data)
```

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3> python Task-2.py

Sorted array: [11, 12, 22, 25, 34, 64, 90]

Task 3: Quick Sort and Merge Sort Comparison

- Task: Implement Quick Sort and Merge Sort using recursion.
- Instructions:
- Provide AI with partially completed functions for recursion.
- Ask AI to complete the missing logic and add docstrings.
- Compare both algorithms on random, sorted, and reverse-sorted lists.
- Expected Output:
- Working Quick Sort and Merge Sort implementations.
- AI-generated explanation of average, best, and worst-case complexities.

The screenshot shows a VS Code editor with a Python file named `Task-3.py`. The script defines a function `compare_algorithms` that generates three input lists: 'random', 'sorted', and 'reverse-sorted'. It then compares the execution time of `quicksort` and `merge_sort` on these inputs. The terminal output shows the results for a list size of 1000, indicating that the 'sorted' input is the fastest, followed by 'reverse-sorted', and 'random' is the slowest.

```
def compare_algorithms(size=1000):
    inputs = {
        "random": [random.randint(0, size) for _ in range(size)],
        "sorted": list(range(size)),
        "reverse-sorted": list(range(size, 0, -1)),
    }
    print("\nComparison of sorting algorithms:")
    print(f"Input type: {size} Quick Sort time (s): {22} Merge Sort time (s): {22}")

    for label, values in inputs.items():
        quick_sorted, quick_time = measure_sort_time(quicksort, values)
        merge_sorted, merge_time = measure_sort_time(merge_sort, values)

        # Verify both algorithms produce the same result and the result is sorted.
        assert quick_sorted == merge_sorted == sorted(values)

        print(f"{label: <15} {quick_time: >22.6f} {merge_time: >22.6f}")

if __name__ == "__main__":
    sample_data = [33, 10, 55, 71, 29, 8, 90]
    print("Original list:", sample_data)

    sorted_with_quick = quicksort(sample_data)
    print("Quick sort result:", sorted_with_quick)

    sorted_with_merge = merge_sort(sample_data)
    print("Merge sort result:", sorted_with_merge)

    compare_algorithms(size=2000)
```

Terminal Output:

```
Comparison of sorting algorithms:
Input type      Quick Sort time (s)  Merge Sort time (s)
random          0.001768            0.001889
sorted          0.001553            0.001314
reverse-sorted  0.001060            0.001373
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store’s inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

```
File Edit Selection View Go Run Terminal Help
2303A52305 AI A-C Ass-12.3

EXPLORER
2303A52305 AI A-C...
AIAC_assignment_12.3_2305.docx
Task-1.py
Task-2.py
Task-3.py
Task-4.py
Task-5.py

Task-4.py >...
56 def merge(left: List[Product], right: List[Product], key, descending: bool) -> List[Product]:
73     merged.extend(right[j:])
74     return merged
75
76
77 if __name__ == "__main__":
78     sample_products = [
79         Product(product_id="P1001", name="Widget", price=9.99, stock_quantity=120),
80         Product(product_id="P1002", name="Gadget", price=14.99, stock_quantity=80),
81         Product(product_id="P1003", name="Widget", price=9.99, stock_quantity=50),
82         Product(product_id="P1004", name="Thingamajig", price=24.99, stock_quantity=200),
83         Product(product_id="P1005", name="Doodad", price=4.99, stock_quantity=30),
84     ]
85
86     inventory = Inventory(sample_products)
87
88     print("Search by ID P1002:")
89     product = inventory.search_by_id("P1002")
90     print(product)
91
92     print("\nSearch by name 'widget':")
93     for match in inventory.search_by_name("widget"):
94         print(match)
95
96     print("\nProducts sorted by price ascending:")
97     for item in inventory.sort_by_price():
98         print(item)
99
100     print("\nProducts sorted by stock quantity descending:")
101     for item in inventory.sort_by_quantity(descending=True):
102         print(item)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
TERMINAL
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3> cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3'; & 'c:\Users\kat
Product(product_id='P1002', name='Gadget', price=14.99, stock_quantity=80)
Product(product_id='P1004', name='Thingamajig', price=24.99, stock_quantity=200)

Products sorted by stock quantity descending:
Product(product_id='P1004', name='Thingamajig', price=24.99, stock_quantity=200)
Product(product_id='P1001', name='Widget', price=9.99, stock_quantity=120)
Product(product_id='P1002', name='Gadget', price=14.99, stock_quantity=80)
Product(product_id='P1003', name='Widget', price=9.99, stock_quantity=50)
Product(product_id='P1005', name='Doodad', price=4.99, stock_quantity=30)
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3> |
```

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by percentage change.
- Implement a search function that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with Heap Sort and searching with Hash Maps.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

The screenshot shows a VS Code editor window with a Python file named `Task-5.py`. The script defines a function `print_bottom_stocks` and a `main` function. The `main` function generates sample stock data, sorts it by percentage change, and prints the top and bottom performing stocks. The terminal output shows the execution of the script, displaying the top and bottom performing stocks.

```
123
124 def print_bottom_stocks(sorted_stocks: List[Stock], bottom_n: int = 5) -> None:
125     """Print the worst-performing stocks based on percentage change."""
126     print(f"\nBottom {bottom_n} stocks by percentage change:")
127     print(f"{'Symbol':<8}{'Open':>8}{'Close':>8}{'Change %':>12}")
128     for stock in sorted_stocks[-bottom_n:]:
129         print(f"{stock.symbol:<8}{stock.opening_price:>8.2f}{stock.closing_price:>8.2f}{stock.percentage_change:>12.2f}")
130
131
132
133 def main() -> None:
134     sample_stocks = generate_stock_data(2000)
135     symbol_index = build_symbol_index(sample_stocks)
136
137     sorted_heap = heap_sort(sample_stocks)
138     print_top_stocks(sorted_heap, top_n=5)
139     print_bottom_stocks(sorted_heap, bottom_n=5)
140
141     benchmark_sorting(sample_stocks)
142
143     search_samples = [sample_stocks[i].symbol for i in range(0, 50, 10)]
144     benchmark_search(symbol_index, search_samples)
145
146     query = sample_stocks[0].symbol
147     found = search_stock(query, symbol_index)
148     print(f"\nInstant search result for symbol {query}:")
149     print(found)
150
151 if __name__ == "__main__":
152     main()
```

Terminal Output:

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3> cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3'; & 'c:\Users\katku\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' -- "c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3\Task-5.py"
Symbol Open Close Change %
MEK 158.86 189.55 19.92
HAC 197.74 237.06 19.88
CEK 320.44 384.12 19.87
FJS 261.89 313.83 19.83
NON 170.91 204.78 19.82
```